



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

Síncronas y Asincrónicas

Presentado a: Instructora Yoli Alexandra Guevara
Por Aprendiz: Sara Isabel Campo Calapsu
Ficha: 3312932
Competencia: FRONTED

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día 14 de 09 del año 2026



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Tabla de Contenido

Contenido

1. Investigación	4
1.1 ¿Qué es una función síncrona?	4
1.2 ¿Qué es una función asíncrona?	4
1.3 ¿Cómo se ejecuta una función síncrona?	4
1.4 ¿Cómo se ejecuta una función asíncrona?	4
1.5 ¿Qué significa que una operación sea bloqueante?	5
1.6 ¿Qué significa que una operación sea no bloqueante?	5
1.7 ¿Qué es una Promise?	5
1.8 ¿Para que sirve async y await?	5
1.9 ¿Qué problemas pueden solucionarse utilizando programación asíncrona? ..	5
2. Tabla Comparativa	6
3. Comprendiendo el funcionamiento	6
3.1 ¿Qué mensaje aparece primero?	7
3.2 ¿Qué mensaje aparece segundo?	7
3.3 ¿Qué mensaje aparece tercero?	7
3.4 ¿La función saludar() debe terminar para continuar con la siguiente instrucción?	7
3.5 ¿Este código es síncrono o asíncrono?	7
4. Analizando una función asíncrona	7
4.1 ¿Qué mensaje aparece primero?	7
4.2 ¿Qué mensaje aparece inmediatamente después?	7
4.3 ¿Qué mensaje aparece aproximadamente después de dos segundos?	8
4.4 ¿Por qué “Fin” aparece antes de “Proceso terminado”?	8
4.5 ¿El programa se detiene durante los 2 segundos?	8
4.6 ¿Qué características de la programación asíncrona se puede observar?	8
5. Prediga el resultado	8



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

5.1 ¿Por qué B no aparece inmediatamente después de A aunque setTimeout() sea 0?	9
6. Funciones con async y await	9
6.1 ¿Qué devuelve obtenerUsuario?	9
6.2 ¿Qué representa resolve()?	10
6.3 ¿Qué función está definida como async?	10
6.4 ¿Qué hace await?	10
6.5 ¿Qué ocurre mientras se esperan los 2 segundos?	10
6.6 ¿Por qué async y await facilitan la lectura del código asíncrono?	10
6.7 ¿Qué ocurriría si se eliminara await?	10
7. ¿Cuándo utilizar cada una?	10
8. Situación problema	11
8.1 ¿Utilizaría una función síncrona o asíncrona?	11
8.2 ¿Por qué?	11
8.3 ¿Qué problema podría presentarse si la aplicación bloqueara completamente la interfaz mientras espera la respuesta?	12
8.4 ¿Qué tecnologías de JavaScript podrían utilizarse?	12
9. Reto práctico	12
Requisito	13
10. Reflexión final	14
10.1 ¿Cuál es la principal diferencia entre una función síncrona y una asíncrona?	14
10.2 ¿Por qué las operaciones que dependen de un servidor normalmente se manejan de forma asíncrona?	14
10.3 ¿Qué ventaja ofrece async/await frente a trabajar directamente con múltiples then()?	14
10.4 ¿Una función asíncrona significa necesariamente que se ejecuta en otro hilo? Explique	14
10.5 Mencione tres situaciones reales en las que utilizaría programación asíncrona	14



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

1. Investigación

1.1 ¿Qué es una función síncrona?

Es la que ejecuta sus instrucciones una después de otra y en orden. El programa debe terminar una operación antes de continuar con la siguiente.

Por ejemplo, si tenemos tres instrucciones 1, 2 y 3, primero se ejecuta 1, después 2 y por último 3.

1.2 ¿Qué es una función asíncrona?

Es la que permite iniciar una operación que puede tardar un tiempo sin detener completamente la ejecución de las demás instrucciones y es muy útil cuando se va a trabajar con operaciones que dependen de algo externo, por ejemplo, en consultar una API, esperar una respuesta de un servidor, consultar una base de datos y leer un archivo. Así puede seguir ejecutando otras instrucciones mientras espera que termine la operación.

1.3 ¿Cómo se ejecuta una función síncrona?

Una función síncrona se ejecuta de manera secuencial, es decir, una instrucción espera a que termine la anterior, ejemplo:

```
console.log("Inicio");  
function saludar() {  
    console.log("Hola aprendiz");  
}
```

```
saludar();
```

```
console.log("Fin");
```

Primero se ejecuta "Inicio", después la función saludar() y finalmente "Fin".

1.4 ¿Cómo se ejecuta una función asíncrona?

Cuando inicia una operación que puede terminar posteriormente, mientras esa operación está esperando, JavaScript puede continuar con otras instrucciones, por ejemplo

```
console.log("Inicio");
```

```
setTimeout(() => {
```

```
    console.log("Proceso terminado");
```



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

```
}, 2000); console.log("Fin");
```

Esto mostrara primero el inicio, después el fin y por ultimo el Proceso terminado mediante el `setTimeout()` que programa el código después del tiempo indicado, en este caso unos 2 segundos.

1.5 ¿Qué significa que una operación sea bloqueante?

Que impide que el programa continúe con otras instrucciones hasta que la ejecución termine, por ejemplo, si una página web tuviera que esperar completamente una respuesta del servidor antes de poder continuar, la interfaz podría quedarse sin responder durante ese tiempo.

1.6 ¿Qué significa que una operación sea no bloqueante?

Que permite que el programa continúe realizando otras tareas mientras espera que termine una ejecución, por ejemplo, al hacer una consulta a un servidor de forma asíncrona la página puede continuar funcionando mientras que espera la respuesta.

1.7 ¿Qué es una Promise?

Es un objeto de JavaScript que representa el resultado futuro de una operación asíncrona y que se pueden encontrar en tres estados que son el Pending que es la operación que se está procesando, el Fulfilled que es la ejecución a terminado correctamente, y el Rejected que es la ejecución a terminado con un error.

1.8 ¿Para que sirve `async` y `await`?

`Async` sirve para indicar que una función trabaje de forma asíncrona y que devuelva una promise y el `Await` es el que se utiliza dentro de una función para esperar el resultado de una promise antes de continuar con la siguiente instrucción de esa función.

1.9 ¿Qué problemas pueden solucionarse utilizando programación asíncrona?

La programación asíncrona permite manejar mejores ejecuciones que pueden tardar en completarse evitando bloquear innecesariamente la página web, por ejemplo, sirve para consultar información de una API, esperar una respuesta de un servidor, consultar una base de datos, leer archivos que pueden tardar en cargarse y realizar operaciones que necesitan esperar un determinado tiempo, de esta manera, la página web puede seguir funcionando y responder al usuario mientras espera que termine la operación.



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

2. Tabla Comparativa

Característica	Funciones Síncronas	Funciones Asíncronas
¿Cómo se ejecutan?	Con una instrucción después de otra.	Cuando se inician una ejecución y pueden continuar con otras instrucciones mientras esperan.
Orden de ejecución	Mantiene el orden del código.	Cuando algunas ejecuciones pueden terminar posteriormente.
¿Bloquean la ejecución?	Se puede bloquear mientras termina la ejecución actual.	Generalmente permiten continuar con otras tareas.
¿Esperan una operación?	Sí, directamente antes de continuar.	Pueden esperar una operación sin tener que bloquear todo el programa.
Tiempo de respuesta	Depende de cuánto se puede tardar la ejecución.	Puede mejorar la respuesta de la aplicación al no bloquearla mientras espera.
Uso de Promise	No es necesario.	Se utilizan para manejar ejecuciones asíncronas.
Uso de async/await	No es necesario.	Se utilizan para facilitar el manejo de Promises.
Ejemplos	Sumar números, calcular una factura, validar datos.	Consultar una API, servidor o base de datos.
¿Cuándo se utilizan?	Cuando la operación es rápida y no necesita esperar recursos externos.	Cuando una operación puede tardar o depende de recursos externos.
Ventajas	Son sencillas y fáciles de seguir.	Evitan bloquear la aplicación y permiten manejar esperas.
Desventajas	Que sea una operación lenta puede detener el flujo de ejecución.	Su lógica puede ser más compleja si no se comprende correctamente.

3. Comprendiendo el funcionamiento

```
console.log("Inicio");  
function saludar() {
```



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

```
    console.log("Hola aprendiz");  
}  
saludar();  
console.log("Fin");
```

3.1 ¿Qué mensaje aparece primero?

El inicio.

3.2 ¿Qué mensaje aparece segundo?

Hola aprendiz.

3.3 ¿Qué mensaje aparece tercero?

Fin

3.4 ¿La función saludar() debe terminar para continuar con la siguiente instrucción?

Sí, Como la función es síncrona, debe terminar su ejecución antes de que JavaScript continúe con console.log("Fin").

3.5 ¿Este código es síncrono o asíncrono?

Es síncrono porque las instrucciones se ejecutan en orden, primero muestra el inicio, luego se llama y ejecuta la función completamente saludar() y por ultimo muestra el Fin.

4. Analizando una función asíncrona

```
console.log("Inicio");  
  
setTimeout(() => {  
    console.log("Proceso terminado");  
}, 2000);  
  
console.log("Fin");
```

4.1 ¿Qué mensaje aparece primero?

El inicio

4.2 ¿Qué mensaje aparece inmediatamente después?

El fin



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

4.3 ¿Qué mensaje aparece aproximadamente después de dos segundos?

Aparece Proceso terminado.

4.4 ¿Por qué “Fin” aparece antes de “Proceso terminado”?

Porque el `setTimeout()` programa la ejecución de “Proceso terminado” para que después de los dos segundos JavaScript continúe con la siguiente instrucción y pueda ejecutar “Fin” antes.

4.5 ¿El programa se detiene durante los 2 segundos?

No, se puede seguir ejecutando otras instrucciones mientras espera que se cumpla el tiempo establecido.

4.6 ¿Qué características de la programación asíncrona se puede observar?

Que una operación puede quedar pendiente mientras el resto del código continúa ejecutándose, permitiendo evitar que una espera bloquee la ejecución.

5. Prediga el resultado

```
console.log("A");  
  
setTimeout(() => {  
    console.log("B");  
}, 0);  
  
console.log("C");  
  
setTimeout(() => {  
    console.log("D");  
}, 1000);  
  
console.log("E");
```

Orden esperado: A, C, E, B, D.

Porque primero se ejecuta el `console.log("A");` por eso aparece la A, con el `setTimeout(() => { console.log("B"); }, 0);` aunque el tiempo sea 0, B no se ejecuta inmediatamente quedaría pendiente para ejecutarse, luego con el `console.log("C");` se programa la D para un segundo después, y por último el



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

`console.log("E");` que es cuando termina la ejecución síncrona pendiente, se ejecuta la B y un segundo después la D.

5.1 ¿Por qué B no aparece inmediatamente después de A aunque `setTimeout()` sea 0?

Porque `setTimeout(..., 0)` no significa ejecutar inmediatamente. Significa que el código queda programado para ejecutarse cuando sea posible, después de que termine la ejecución síncrona que está pendiente, por eso primero aparece A, C, E y después B.

6. Funciones con `async` y `await`

Analice

```
function obtenerUsuario() {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve("Usuario encontrado");  
    }, 2000);  
  });  
}
```

```
async function mostrarUsuario() {  
  
  console.log("Buscando usuario...");  
  
  const resultado = await obtenerUsuario();  
  
  console.log(resultado);  
  
  console.log("Proceso terminado");  
}
```

```
mostrarUsuario();
```

6.1 ¿Qué devuelve `obtenerUusuario`?

Una promise porque dentro de la función se crea y retorna `new Promise(..)` haciendo que se resuelva después de 2 segundos.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

6.2 ¿Qué representa resolve()?

Que la promise termino correctamente y proporciona el resultado resolve("Usuario encontrado"); por lo tanto el resultado de la Promise será "usuario encontrado"

6.3 ¿Qué función está definida como async?

La función async function mostrarUsuario() esta definida como async

6.4 ¿Qué hace await?

Que espera a que la Promise termine para poder obtener su resultado y poder continuar con las siguientes instrucciones de esa función, en este caso const resultado = await obtenerUsuario(); espera hasta que obtenerUsuario() termine.

6.5 ¿Qué ocurre mientras se esperan los 2 segundos?

La función mostrarUsuario() se queda esperando el resultado de la Promise pero sin tener que bloquear completamente todo el JavaScript por esos 2 segundos, cuando la Promise se resuelve continua la ejecución y se muestra Usuario encontrado.

6.6 ¿Por qué async y await facilitan la lectura del código asíncrono?

Porque permiten escribir el código asíncrono de una forma parecida al código secuencial, haciéndose más fácil de entender qué operación se está esperando y qué instrucciones se ejecutan después.

6.7 ¿Qué ocurriría si se eliminara await?

Si se elimina el await const resultado = obtenerUsuario(); el resultado recibiría la Promise no directamente el texto Usuario encontrado, por lo tanto console.log(resultado) mostraría la Promise o su estado, en lugar del resultado final esperado.

7. ¿Cuándo utilizar cada una?

Indique si utilizaría una función **SÍNCRONA (S)** o **ASÍNCRONA (A)**.

Situación	S / A	¿Por qué?
Sumar dos números	S	es una operación rápida que no necesita esperar información externa.
Validar que un campo no esté vacío	S	la validación se puede realizar inmediatamente con los datos disponibles.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

Situación	S / A	¿Por qué?
Consultar información de una API	A	hay que esperar una respuesta de un servidor.
Consultar una base de datos	A	la aplicación debe esperar la respuesta de la base de datos.
Esperar una respuesta de un servidor	A	la respuesta puede tardar y no conviene bloquear la aplicación.
Calcular el total de una factura	S	el cálculo se realiza con la información que ya tenemos disponible.
Leer información de un archivo de manera asíncrona	A	la lectura puede tardar y se puede realizar sin bloquear la aplicación.
Realizar una animación con retraso	A	se necesita esperar un tiempo antes de realizar una acción.
Procesar una operación que depende de una API	A	primero se necesita obtener la información de la API.
Mostrar el resultado de una operación matemática	S	el resultado se puede obtener directamente después del cálculo.

8. Situación problema

Una empresa necesita desarrollar una aplicación web para consultar información de sus clientes.

Cuando el usuario presiona “**Consultar cliente**”, la aplicación debe enviar una solicitud al servidor, esperar la respuesta, recibir los datos del cliente y mostrar la información.

8.1 ¿Utilizaría una función síncrona o asíncrona?

Utilizaría una función asíncrona.

8.2 ¿Por qué?

Porque la aplicación web necesita enviar una solicitud a un servidor y esperar una respuesta, aunque la ejecución puede tardar algunos segundos es mejor realizarla de manera asíncrona para evitar bloquear la aplicación.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

8.3 ¿Qué problema podría presentarse si la aplicación bloqueara completamente la interfaz mientras espera la respuesta?

La interfaz podría quedarse congelada o sin responder mientras espera al servidor y podrían pensar que la aplicación web se dañó porque no podría interactuar normalmente con la página.

8.4 ¿Qué tecnologías de JavaScript podrían utilizarse?

Promise: porque representa el resultado futuro de una ejecución asíncrona, en este caso puede representar la respuesta que llegará desde el servidor.

Async: permite definir una función que trabaja con operaciones asíncronas y devuelve una Promise, ejemplo:

```
async function consultarCliente() {  
}
```

Await: permite esperar el resultado de una Promise dentro de una función async, ejemplo:

```
const respuesta = await fetch(url);
```

fetch: permite realizar solicitudes HTTP desde JavaScript, por ejemplo para solicitar información a un servidor o API, ejemplo:

```
const respuesta = await fetch(url);
```

En si estas herramientas permiten realizar la consulta al servidor, esperar su respuesta y mostrar los datos del cliente.

9. Reto practico

Construya una función que simule la consulta de un usuario:

```
function consultarUsuario() {  
  
    // Simular consulta al servidor  
  
}
```

La función debe tardar 3 segundos en responder.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

Después debe mostrar:

Consultando usuario...

Usuario encontrado

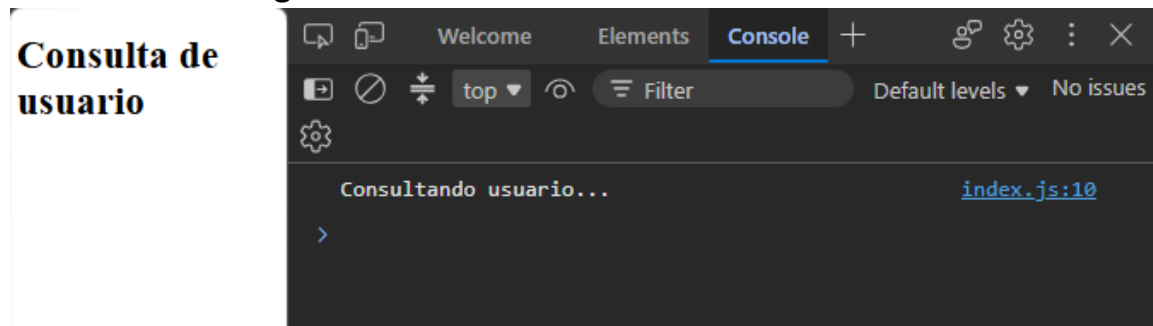
Proceso terminado

Requisito

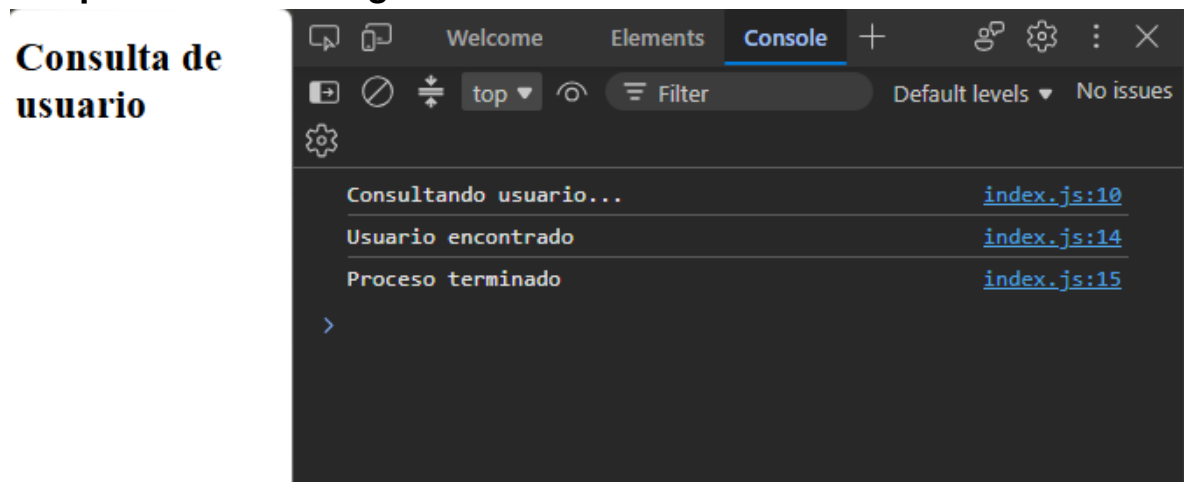
Utilice:

- Promise
- setTimeout()
- async
- await

Evidencia codigo



Después de los 3 segundos





**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

10. Reflexión final

10.1 ¿Cuál es la principal diferencia entre una función síncrona y una asíncrona?

La principal diferencia es que una función síncrona ejecuta las instrucciones en orden y espera a que termine cada operación antes de continuar, mientras que una función asíncrona permite manejar operaciones que pueden tardar sin bloquear completamente la ejecución del programa.

10.2 ¿Por qué las operaciones que dependen de un servidor normalmente se manejan de forma asíncrona?

Porque la respuesta del servidor puede tardar. Si se utilizara una operación completamente bloqueante, la aplicación podría quedarse esperando y la interfaz podría dejar de responder, con la programación asíncrona, la aplicación puede continuar funcionando mientras espera la respuesta.

10.3 ¿Qué ventaja ofrece `async/await` frente a trabajar directamente con múltiples `then()`?

La principal ventaja es que `async` y `await` hacen que el código sea más fácil de leer y entender, especialmente cuando existen varias operaciones que deben ejecutarse una después de otra.

10.4 ¿Una función asíncrona significa necesariamente que se ejecuta en otro hilo? Explique

No, que una función sea asíncrona no significa necesariamente que se ejecute en otro hilo, si no que la asincronía permite gestionar operaciones que deben esperar, como respuestas de servidores o temporizadores, sin bloquear la ejecución principal mientras están pendientes.

10.5 Mencione tres situaciones reales en las que utilizaría programación asíncrona

Utilizaría programación asíncrona en Consultar una API porque se debe esperar la respuesta del servidor, consultar una base de datos porque la información puede tardar en llegar y en enviar o recibir información de un servidor porque la comunicación puede tomar un tiempo.